HASHMAP • HASHSET • MAP

Java Collections Framework • Concepts • Formulas • IntelliJ Project • Runnable Interview Code

A practical guide to choosing the right collection, writing frequency maps, set-based lookups, grouping, indexing, and solving common interview patterns.

Java | IntelliJ IDEA | Product-based & high-tech interviews

1. First understand the relationship

Map is an interface that stores key → value pairs. HashMap is a hash-table-based implementation of Map. HashSet stores unique elements and is backed internally by a HashMap in standard Java implementations.

• Map<K,V>: keys are unique; each key maps to at most one value. Values may repeat.

• HashMap<K,V>: fast average-case put/get/remove; permits one null key and multiple null values; does not guarantee iteration order.

• HashSet<E>: stores unique elements; add returns false if the element already exists; permits one null element; does not guarantee iteration order.

• LinkedHashMap / LinkedHashSet: preserve insertion order.

• TreeMap / TreeSet: maintain sorted key/element order; typical operations O(log n).

• ConcurrentHashMap: thread-safe concurrent map; does not allow null keys or values.

Imports

```
import java.util.*;
```

IntelliJ project structure

```
HashCollections/
■■■ src/
    ■■■ collections/
        ■■■ basics/
        ■   ■■■ MapBasics.java
        ■   ■■■ HashMapBasics.java
        ■   ■■■ HashSetBasics.java
        ■   ■■■ LinkedHashMapBasics.java
        ■   ■■■ TreeMapBasics.java
        ■■■ frequency/
        ■   ■■■ FrequencyCounter.java
        ■   ■■■ FirstUniqueCharacter.java
        ■   ■■■ MajorityElement.java
        ■   ■■■ TopKFrequent.java
        ■■■ lookup/
        ■   ■■■ TwoSum.java
        ■   ■■■ ContainsDuplicate.java
        ■   ■■■ IntersectionArrays.java
        ■   ■■■ LongestConsecutive.java
        ■■■ prefixsum/
        ■   ■■■ SubarraySumEqualsK.java
        ■   ■■■ CountSubarraysDivisibleByK.java
        ■   ■■■ ZeroSumSubarray.java
        ■■■ strings/
        ■   ■■■ GroupAnagrams.java
        ■   ■■■ IsomorphicStrings.java
        ■   ■■■ WordPattern.java
        ■■■ design/
            ■■■ LRUCache.java
```

Create the package exactly as shown, then create each public class in its own same-named .java file. The code below includes package and import declarations. For Java 8 compatibility, the examples avoid newer convenience APIs where practical.

2. Hashing: the core idea

A hash function converts a key into an integer hash code. A hash table uses that hash to select a bucket/index. Different keys can land in the same bucket; this is a collision. Java HashMap handles collisions using bucket structures and, under certain conditions, tree bins.

Core operations and average complexity

Operation	HashMap / HashSet average	Worst-case note
put / add	O(1)	Can degrade with heavy collisions; tree bins improve collision handling
get / contains	O(1)	Same collision caveat
remove	O(1)	Same collision caveat
Iteration	O(n)	Depends on table capacity and size
TreeMap / TreeSet operations	O(log n)	Sorted order

HashMap index intuition

```
int hash = key.hashCode();
int bucketIndex = (tableLength - 1) & spread(hash);
```

This is conceptual, not code you should reimplement. Java's actual implementation includes hash spreading, resizing, load-factor management, and collision handling. Default load factor is 0.75; capacity is commonly a power of two.

equals() and hashCode() contract

• If a.equals(b) is true, a.hashCode() and b.hashCode() must be equal.

• Equal hash codes do not prove objects are equal: collisions are allowed.

• Override equals() and hashCode() together for custom key classes.

• Do not mutate fields used by equals/hashCode while an object is a map key or set element; lookup may stop working as expected.

3. Map interface — essential methods

File: src/collections/basics/MapBasics.java

Concept / formula: Map<K,V> is the abstraction. Common methods: put, get, getOrDefault, putIfAbsent, containsKey, containsValue, remove, replace, keySet, values, entrySet, computeIfAbsent, merge.

```
package collections.basics;

import java.util.*;

public class MapBasics {
    public static void main(String[] args) {
        Map<String, Integer> marks = new HashMap<>();

        marks.put("Asha", 90);
        marks.put("Ravi", 82);
        marks.put("Asha", 95); // replaces old value

        System.out.println(marks.get("Asha")); // 95
        System.out.println(marks.getOrDefault("Neha", 0)); // 0
        System.out.println(marks.containsKey("Ravi")); // true

        marks.putIfAbsent("Neha", 88);
        marks.replace("Ravi", 85);
        marks.remove("Asha");

        for (Map.Entry<String, Integer> e : marks.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue());
        }

        marks.forEach((name, score) ->
            System.out.println(name + ": " + score));
    }
}
```

Complexity: put/get/remove average O(1) for HashMap; iteration O(n).

4. HashMap — common operations

File: src/collections/basics/HashMapBasics.java

Concept / formula: HashMap stores key-value pairs. Updating an existing key replaces its value; it does not create a duplicate key.

```
package collections.basics;

import java.util.*;

public class HashMapBasics {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();

        map.put(101, "Java");
        map.put(102, "Python");
        map.put(101, "Advanced Java");

        System.out.println(map.get(101));
        System.out.println(map.get(999)); // null
        System.out.println(map.getOrDefault(999, "Not found"));

        if (map.containsKey(102)) {
            System.out.println("Key exists");
        }

        map.remove(102);

        for (Integer key : map.keySet()) {
            System.out.println(key + " = " + map.get(key));
        }
```

```
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Complexity: Average O(1) for put/get/containsKey/remove.

5. HashSet — uniqueness and membership

File: src/collections/basics/HashSetBasics.java

Concept / formula: HashSet keeps only unique values. add(x) returns true only when x was not already present. Use it when membership/uniqueness matters and key-value mapping is unnecessary.

```
package collections.basics;

import java.util.*;

public class HashSetBasics {
    public static void main(String[] args) {
        Set<Integer> set = new HashSet<>();

        System.out.println(set.add(10)); // true
        System.out.println(set.add(20)); // true
        System.out.println(set.add(10)); // false

        System.out.println(set.contains(20)); // true
        set.remove(10);

        for (int value : set) {
            System.out.println(value);
        }
    }
}
```

Complexity: Average O(1) for add/contains/remove; O(n) iteration.

6. LinkedHashMap — insertion order

File: src/collections/basics/LinkedHashMapBasics.java

Concept / formula: LinkedHashMap preserves insertion order by default. It can also be configured for access order, which is useful in LRU cache designs.

```
package collections.basics;

import java.util.*;

public class LinkedHashMapBasics {
    public static void main(String[] args) {
        Map<Integer, String> map = new LinkedHashMap<>();
        map.put(3, "C");
        map.put(1, "A");
        map.put(2, "B");

        System.out.println(map); // insertion order: 3, 1, 2
    }
}
```

Complexity: Average O(1) basic map operations; iteration follows insertion order.

7. TreeMap — sorted keys

File: src/collections/basics/TreeMapBasics.java

Concept / formula: TreeMap sorts keys by natural ordering or a supplied Comparator. Use firstKey, lastKey, floorKey, ceilingKey, lowerKey and higherKey for ordered queries.

```
package collections.basics;

import java.util.*;

public class TreeMapBasics {
    public static void main(String[] args) {
        TreeMap<Integer, String> map = new TreeMap<>();
        map.put(30, "C");
        map.put(10, "A");
        map.put(20, "B");

        System.out.println(map); // {10=A, 20=B, 30=C}
        System.out.println(map.firstKey()); // 10
        System.out.println(map.lastKey());  // 30
        System.out.println(map.floorKey(25)); // 20
        System.out.println(map.ceilingKey(25)); // 30
    }
}
```

Complexity: O(log n) for get/put/remove and navigational operations.

8. Frequency map — the most-used interview formula

Formula: freq[x] = freq[x] + 1. In Java: map.put(x, map.getOrDefault(x, 0) + 1).

Frequency Counter

File: src/collections/frequency/FrequencyCounter.java

Concept / formula: Count how many times each value occurs. Use HashMap when values are not limited to a small index range.

```
package collections.frequency;

import java.util.*;

public class FrequencyCounter {
    static Map<Integer, Integer> frequency(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) {
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }
        return freq;
    }

    public static void main(String[] args) {
        int[] nums = {2, 3, 2, 5, 3, 2};
        System.out.println(frequency(nums)); // frequencies: 2->3, 3->2, 5->1
    }
}
```

Complexity: Expected O(n) time, O(k) space where k is number of distinct values.

9. First Non-Repeating Character

File: src/collections/frequency/FirstUniqueCharacter.java

Concept / formula: Count first, then scan original order. The first character with frequency 1 is the answer.

```
package collections.frequency;

import java.util.*;

public class FirstUniqueCharacter {
    static int firstUniqueIndex(String s) {
        Map<Character, Integer> freq = new HashMap<>();
        for (char ch : s.toCharArray())
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);

        for (int i = 0; i < s.length(); i++) {
            if (freq.get(s.charAt(i)) == 1) return i;
        }
        return -1;
    }

    public static void main(String[] args) {
        System.out.println(firstUniqueIndex("leetcode")); // 0
    }
}
```

Complexity: O(n) time, O(k) space.

10. Majority Element

File: src/collections/frequency/MajorityElement.java

Concept / formula: Majority means frequency strictly greater than n/2. HashMap counting is direct; Boyer-Moore can solve it in O(1) extra space when the existence guarantee applies.

```
package collections.frequency;

import java.util.*;

public class MajorityElement {
    static int majority(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) {
            int count = freq.getOrDefault(x, 0) + 1;
            freq.put(x, count);
            if (count > nums.length / 2) return x;
        }
        return -1;
    }

    public static void main(String[] args) {
```

```
        System.out.println(majority(new int[]{2,2,1,1,1,2,2})); // 2
    }
}
```

Complexity: Expected O(n) time, O(k) space.

11. Top K Frequent Elements

File: src/collections/frequency/TopKFrequent.java

Concept / formula: Build frequencies, then use a min-heap of size k. The heap root is the least frequent among the retained candidates.

```
package collections.frequency;

import java.util.*;

public class TopKFrequent {
    static int[] topK(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums)
            freq.put(x, freq.getOrDefault(x, 0) + 1);

        PriorityQueue<Integer> heap = new PriorityQueue<>(
            (a, b) -> Integer.compare(freq.get(a), freq.get(b))
        );

        for (int x : freq.keySet()) {
            heap.offer(x);
            if (heap.size() > k) heap.poll();
        }

        int[] answer = new int[Math.min(k, heap.size())];
        for (int i = answer.length - 1; i >= 0; i--)
            answer[i] = heap.poll();
        return answer;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            topK(new int[]{1,1,1,2,2,3}, 2))); // [1, 2]
    }
}
```

Complexity: O(n + u log k) time, O(u+k) space; u=distinct values.

12. HashSet patterns: fast membership

Contains Duplicate

File: src/collections/lookup/ContainsDuplicate.java

Concept / formula: If set.add(x) returns false, x has already appeared. This is a compact duplicate-detection pattern.

```
package collections.lookup;

import java.util.*;

public class ContainsDuplicate {
    static boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) {
            if (!seen.add(x)) return true;
        }
        return false;
    }

    public static void main(String[] args) {
        System.out.println(containsDuplicate(new int[]{1,2,3,1})); // true
    }
}
```

Complexity: Expected O(n) time, O(n) space.

13. Two Sum

File: src/collections/lookup/TwoSum.java

Concept / formula: Formula: needed = target - current. Store value → index. Check the complement before inserting current value so an element is not paired with itself.

```
package collections.lookup;

import java.util.*;

public class TwoSum {
    static int[] twoSum(int[] nums, int target) {
```

```
        Map<Integer, Integer> index = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (index.containsKey(need)) {
                return new int[]{index.get(need), i};
            }
            index.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            twoSum(new int[]{2,7,11,15}, 9))); // [0, 1]
    }
}
```

Complexity: Expected O(n) time, O(n) space.

14. Intersection of Two Arrays

File: src/collections/lookup/IntersectionArrays.java

Concept / formula: Use a set for membership. For unique intersection, remove a match after recording it or build a second result set.

```
package collections.lookup;

import java.util.*;

public class IntersectionArrays {
    static int[] intersection(int[] a, int[] b) {
        Set<Integer> set = new HashSet<>();
        for (int x : a) set.add(x);

        Set<Integer> result = new HashSet<>();
        for (int x : b) {
            if (set.contains(x)) result.add(x);
        }

        int[] answer = new int[result.size()];
        int i = 0;
        for (int x : result) answer[i++] = x;
        return answer;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            intersection(new int[]{1,2,2,1}, new int[]{2,2}))); // [2]
    }
}
```

Complexity: Expected O(n+m) time, O(n+m) space.

15. Longest Consecutive Sequence

File: src/collections/lookup/LongestConsecutive.java

Concept / formula: Put all values in a set. Start a sequence only when x-1 is absent; then count x, x+1, ... This prevents rescanning every sequence from its middle.

```
package collections.lookup;

import java.util.*;

public class LongestConsecutive {
    static int longest(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums) set.add(x);

        int best = 0;
        for (int x : set) {
            if (!set.contains(x - 1)) {
                int current = x;
                int length = 1;
                while (set.contains(current + 1)) {
                    current++;
                    length++;
                }
                best = Math.max(best, length);
            }
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longest(new int[]{100,4,200,1,3,2})); // 4
    }
}
```

Complexity: Expected O(n) time, O(n) space. For unrestricted int extremes, guard x-1/x+1 overflow using long or boundary checks.

16. Prefix sum + HashMap: high-value interview pattern

Subarray sum equals K: if prefixSum[j] - prefixSum[i] = k, then prefixSum[i] = prefixSum[j] - k. Store counts of previous prefix sums.

Subarray Sum Equals K

File: src/collections/prefixsum/SubarraySumEqualsK.java

Concept / formula: Initialize map with prefix sum 0 occurring once. At each element, add map[prefix-k] to answer, then record current prefix. This works with negative numbers too.

```
package collections.prefixsum;

import java.util.*;

public class SubarraySumEqualsK {
    static int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        count.put(0, 1);

        int prefix = 0, answer = 0;
        for (int x : nums) {
            prefix += x;
            answer += count.getOrDefault(prefix - k, 0);
            count.put(prefix, count.getOrDefault(prefix, 0) + 1);
        }
        return answer;
    }

    public static void main(String[] args) {
        System.out.println(subarraySum(new int[]{1,1,1}, 2)); // 2
    }
}
```

Complexity: Expected O(n) time, O(n) space. Use long for prefix and answer if constraints can exceed int.

17. Count Subarrays Divisible by K

File: src/collections/prefixsum/CountSubarraysDivisibleByK.java

Concept / formula: If two prefix sums have the same remainder modulo k, the subarray between them is divisible by k. Normalize negative remainders with Math.floorMod.

```
package collections.prefixsum;

import java.util.*;

public class CountSubarraysDivisibleByK {
    static int count(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        freq.put(0, 1);

        int prefix = 0, answer = 0;
        for (int x : nums) {
            prefix += x;
            int rem = Math.floorMod(prefix, k);
            answer += freq.getOrDefault(rem, 0);
            freq.put(rem, freq.getOrDefault(rem, 0) + 1);
        }
        return answer;
    }

    public static void main(String[] args) {
        System.out.println(count(new int[]{4,5,0,-2,-3,1}, 5)); // 7
    }
}
```

Complexity: Expected O(n) time, O(min(n,k)) to O(n) map space. k must be non-zero.

18. Zero Sum Subarray Exists

File: src/collections/prefixsum/ZeroSumSubarray.java

Concept / formula: A zero-sum subarray exists if a prefix sum repeats. Include initial prefix sum 0 to detect a subarray starting at index 0.

```
package collections.prefixsum;

import java.util.*;

public class ZeroSumSubarray {
    static boolean exists(int[] nums) {
```

```
        Set<Integer> seen = new HashSet<>();
        int prefix = 0;
        seen.add(0);

        for (int x : nums) {
            prefix += x;
            if (seen.contains(prefix)) return true;
            seen.add(prefix);
        }
        return false;
    }

    public static void main(String[] args) {
        System.out.println(exists(new int[]{4,2,-3,1,6})); // true
    }
}
```

Complexity: Expected O(n) time, O(n) space.

19. String + Map/Set patterns

Group Anagrams

File: src/collections/strings/GroupAnagrams.java

Concept / formula: Anagrams share the same sorted-character signature. Map signature → list of original words. For lowercase English letters, a 26-count signature can avoid sorting.

```
package collections.strings;

import java.util.*;

public class GroupAnagrams {
    static List<List<String>> group(String[] words) {
        Map<String, List<String>> groups = new HashMap<>();

        for (String word : words) {
            char[] chars = word.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);

            groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
        }
        return new ArrayList<>(groups.values());
    }

    public static void main(String[] args) {
        String[] words = {"eat","tea","tan","ate","nat","bat"};
        System.out.println(group(words));
    }
}
```

Complexity: O(n·m log m) time for n words of average length m; O(total characters) storage.

20. Isomorphic Strings

File: src/collections/strings/IsomorphicStrings.java

Concept / formula: Two strings are isomorphic if each character in one maps consistently to exactly one character in the other, with no two source characters mapping to the same target character. Use two maps or two fixed arrays.

```
package collections.strings;

import java.util.*;

public class IsomorphicStrings {
    static boolean isomorphic(String s, String t) {
        if (s.length() != t.length()) return false;

        Map<Character, Character> forward = new HashMap<>();
        Map<Character, Character> backward = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {
            char a = s.charAt(i), b = t.charAt(i);

            if (forward.containsKey(a) && forward.get(a) != b) return false;
            if (backward.containsKey(b) && backward.get(b) != a) return false;

            forward.put(a, b);
            backward.put(b, a);
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isomorphic("egg", "add")); // true
        System.out.println(isomorphic("foo", "bar")); // false
    }
}
```

Complexity: O(n) time, O(k) space.

21. Word Pattern

File: src/collections/strings/WordPattern.java

Concept / formula: Map pattern character → word plus a set of already-used words enforces a one-to-one mapping.

```
package collections.strings;

import java.util.*;

public class WordPattern {
    static boolean matches(String pattern, String sentence) {
        String[] words = sentence.split(" ");
        if (pattern.length() != words.length) return false;

        Map<Character, String> map = new HashMap<>();
        Set<String> used = new HashSet<>();

        for (int i = 0; i < pattern.length(); i++) {
            char ch = pattern.charAt(i);
            String word = words[i];

            if (map.containsKey(ch)) {
                if (!map.get(ch).equals(word)) return false;
            } else {
                if (used.contains(word)) return false;
                map.put(ch, word);
                used.add(word);
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(matches("abba", "dog cat cat dog")); // true
    }
}
```

Complexity: O(n) expected time, O(n) space.

22. LRU Cache — Map + doubly linked list

LRU = Least Recently Used. Required operations get and put in O(1) average time. A HashMap locates nodes; a doubly linked list maintains recency. Most recently used is near the head; least recently used is near the tail.

LRU Cache

File: src/collections/design/LRUCache.java

Concept / formula: On get: move node to front. On put existing key: update and move to front. On new key at full capacity: remove tail node and its map entry.

```
package collections.design;

import java.util.*;

public class LRUCache {
    private static class Node {
        int key, value;
        Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }

    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0);
    private final Node tail = new Node(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addFirst(Node node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void moveToFront(Node node) {
```

```
        remove(node);
        addFirst(node);
    }

    public int get(int key) {
        Node node = map.get(key);
        if (node == null) return -1;
        moveToFront(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (capacity <= 0) return;

        if (map.containsKey(key)) {
            Node node = map.get(key);
            node.value = value;
            moveToFront(node);
            return;
        }

        if (map.size() == capacity) {
            Node lru = tail.prev;
            remove(lru);
            map.remove(lru.key);
        }

        Node node = new Node(key, value);
        addFirst(node);
        map.put(key, node);
    }

    public static void main(String[] args) {
        LRUCache cache = new LRUCache(2);
        cache.put(1, 10);
        cache.put(2, 20);
        System.out.println(cache.get(1)); // 10
        cache.put(3, 30); // evicts key 2
        System.out.println(cache.get(2)); // -1
        cache.put(4, 40); // evicts key 1
        System.out.println(cache.get(1)); // -1
        System.out.println(cache.get(3)); // 30
        System.out.println(cache.get(4)); // 40
    }
}
```

Complexity: Expected O(1) time for get/put, O(capacity) space.

23. Useful formulas and code templates

Frequency increment

```
map.put(x, map.getOrDefault(x, 0) + 1);
```

Frequency decrement / delete zero

```
int count = map.get(x) - 1;
if (count == 0) map.remove(x);
else map.put(x, count);
```

Group values by key

```
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
```

Merge counts

```
map.merge(key, 1, Integer::sum);
```

Prefix sum lookup

```
answer += freq.getOrDefault(prefix - target, 0);
freq.put(prefix, freq.getOrDefault(prefix, 0) + 1);
```

Set duplicate check

```
if (!seen.add(value)) {
    // value already existed
}
```

Map iteration

```
for (Map.Entry<K, V> entry : map.entrySet()) {
    K key = entry.getKey();
    V value = entry.getValue();
}
```

24. Choosing the correct collection

Need	Choose	Reason
Key → value association	HashMap / Map	Lookup by key
Unique values / membership	HashSet / Set	Duplicates rejected
Preserve insertion order	LinkedHashMap / LinkedHashSet	Predictable iteration
Sorted keys / range queries	TreeMap / TreeSet	Ordered operations
Queue / deque	ArrayDeque	Efficient end operations
Thread-safe high concurrency map	ConcurrentHashMap	Concurrent access support

25. Interview problem bank — high priority

• Two Sum; Contains Duplicate; Valid Anagram; Group Anagrams; Top K Frequent Elements.

• First Unique Character; Majority Element; Ransom Note; Isomorphic Strings; Word Pattern.

• Intersection of Two Arrays I/II; Happy Number; Longest Consecutive Sequence.

• Subarray Sum Equals K; Continuous Subarray Sum; Contiguous Array; Count Number of Nice Subarrays.

• Longest Substring Without Repeating Characters; Minimum Window Substring; Find All Anagrams in a String; Permutation in String.

• Design HashMap; Design HashSet; LRU Cache; LFU Cache; Insert Delete GetRandom O(1).

• Accounts Merge; Evaluate Division; Time Based Key-Value Store; Snapshot Array.

• Pair Sum / 3Sum frequency variants; duplicate within k distance; nearby almost duplicate (TreeSet or buckets).

26. Common mistakes to avoid

• Using map.get(key) and assuming null always means key absent; use containsKey when null values are possible.

• Using == to compare String values; use equals().

• Relying on HashMap or HashSet iteration order; it is not guaranteed.

• Using mutable objects as keys when fields used by equals/hashCode can change.

• Writing a comparator as a.get(x)-b.get(x), which can overflow; use Integer.compare(a,b).

• Using HashSet when frequency counts are required; use HashMap.

• Using HashMap when only membership is needed; HashSet is simpler.

• For prefix sum with negative values, using sliding window blindly; use prefix sum + HashMap when negatives are allowed.

• Assuming worst-case HashMap operations are always O(1); state expected/average complexity accurately.

• Using int for large prefix sums or counts without checking constraints.

27. Final formula sheet

```
FREQUENCY:
freq[x] = freq[x] + 1

TWO SUM:
need = target - x

PREFIX SUM:
prefix[i] = prefix[i-1] + a[i]
subarray(i..j) = prefix[j] - prefix[i-1]

SUBARRAY SUM = K:
count += freq[prefix - K]
freq[prefix]++

ZERO SUM SUBARRAY:
same prefix sum appears twice

SUBARRAY SUM DIVISIBLE BY K:
same normalized remainder appears twice

GROUP ANAGRAMS:
sorted(word) -> group key
(or 26-letter frequency signature)
```

```
LONGEST CONSECUTIVE:
start only if x-1 is absent

HASHSET DUPLICATE:
if (!set.add(x)) duplicate found

LRU:
HashMap(key -> node) + doubly linked list
get/put expected O(1)
```

Coverage note: This volume contains full runnable source code for the core examples listed above and a broad interview problem bank. The problem bank is a practice checklist; it does not claim to include a separate full implementation for every named problem (for example LFU Cache, Insert-Delete-GetRandom, or Minimum Window Substring).